Big 4 Thought-Leadership Intelligence (BTLI)

An end-to-end Python pipeline that collects, cleans and analyses the published thought leadership of Deloitte, EY, KPMG and PwC.

Prepared by Shayan Ponnanna Malchira

1. Summary

Professional-services firms use published research to shape client agendas. Comparing four firms across five topics and 200 documents by hand is slow and subjective. BTLI turns three loosely coupled scripts into one installable, tested Python package with a command-line interface, run manifests and an auto-generated PDF report. While restructuring, I fixed 16 defects; the most serious was that 16 of the 200 source URLs are PDFs, which the original downloader saved as corrupted HTML.

- **Scrape:** requests first, headless Chromium (Playwright) only for pages that look empty; HTML and PDF supported.
- **Extract:** trafilatura for text and tables, BeautifulSoup and pypdf as fallbacks; pages under 150 words flagged.
- **Analyse:** TF-IDF distinctive terms, theme density per 1,000 words, readability, firm similarity, and lexicon validation against human labels.
- **Report:** a PDF with charts, rebuilt from the data on every run.

2. Input data

200 papers (50 per firm, 10 per firm per topic) with the columns Firm, Title, URL, Publish_date and Trend_category. The five topics are AI & Agentic, Finance & Work, Cyber & Risk, M&A & Deals and ESG & Climate.

- 16 URLs are PDFs and 1 URL appears twice; 46 rows have no usable date and the rest use 8+ date formats.
- Seven different hosts, including assets.kpmg.com and legacy Deloitte domains.

3. System design

Module	Responsibility
config.py	Paths, thresholds, theme lexicons, stop-lists
papers.py	CSV loading, date parsing, collision-free file names
scrape.py	HTML/PDF download with retries and browser fallback
extract.py	Text, tables and PDF text with fallbacks and quality flags
analyze.py	Statistics, TF-IDF, theme density, validation, charts
report.py / demo.py	PDF generation / synthetic offline corpus for tests and demos
__main__.py	CLI: scrape | extract | analyze | report | all | demo

Stages communicate through manifests (scrape_manifest.csv, extraction_manifest.csv), so each stage can be run and tested alone.

4. Methodology

- **Collection:** browser-like User-Agent, 3 attempts with exponential back-off, Chromium fallback for JavaScript shells (under 400 visible characters), a random 2–3 s delay between rows, and resume-safe caching.
- **Extraction:** trafilatura removes menus, footers and cookie banners; BeautifulSoup is the fallback; PDFs are read with pypdf; thin papers are excluded.
- **Distinctive terms:** uni- and bi-gram TF-IDF (sublinear tf) over one document per firm, excluding terms present in all four firms and publisher boilerplate.
- **Theme emphasis:** five lexicons, whole-word and case-insensitive, longest-term-first to avoid double counting, reported per 1,000 words.
- **Validation:** per paper, the five theme densities are z-scored and the highest is compared with the human category label.

5. Key engineering improvements

Problem in the original scripts	Fix
PDF URLs saved as corrupt .html files	Detect PDFs by Content-Type / %PDF magic bytes; save as .pdf; extract with pypdf
No retries; a transient 429/503 became a permanent failure	3 attempts with exponential back-off; 4xx errors stop immediately
New Chromium instance per fallback page; networkidle timeouts	One reused browser; wait for domcontentloaded, then best-effort networkidle
File names could collide and skip real papers; dates mis-parsed	Hash-suffixed unique stems; explicit parser for quarter, range and no-date tokens
Lexicon keys did not match CSV categories; overlapping terms double-counted	Keys now equal categories; single longest-first regex; new validation step
Site boilerplate dominated ‘distinctive’ terms	max_df=0.75 and a boilerplate stop-list
Stages coupled through folder arguments and cross-imports	Shared papers.py and manifests; every stage runs and tests alone

6. Results

Theme emphasis by firm (lexicon hits per 1,000 words)

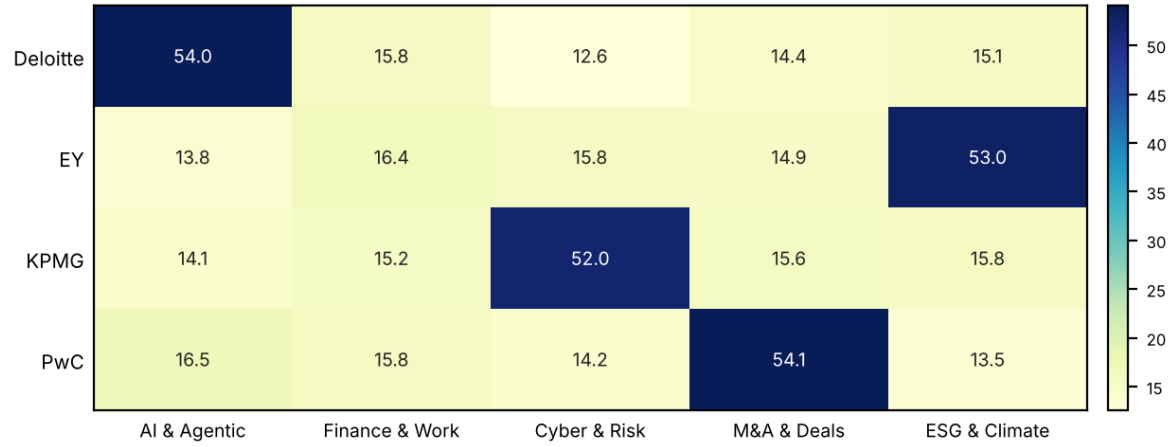


Figure 1 – each firm’s highest-density theme stands out clearly in the demo data

Lexicon validation

Across 192 usable papers the lexicon prediction matches the human category label for **84.4%** of papers (random guessing among five classes: about 20%).

Planted-signal recovery test

Invented marker words inserted into each firm’s texts were ranked for the correct firm in the top 15: **6 of 6 for all four firms (100%)**.

Coverage: 192 of 200 papers passed the 150-word threshold; 8 were flagged for manual review.

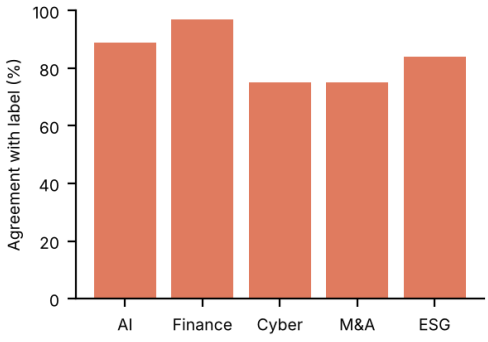


Figure 2 – agreement by category

7. Testing

13 automated pytest tests cover date parsing for every format in the CSV, unique file names for all 200 rows, the lexicon no-double-counting rule, HTML/table/PDF extraction, thin-page flagging, and a full offline pipeline run including the planted-signal check.

8. Limitations

- Only four documents (one per firm) enter TF-IDF, and small differences are not statistically tested.
- Keyword lexicons are a proxy for topics; validation quantifies the error but does not remove it.
- The 200 URLs are curated and balanced, so results describe that sample rather than each firm's full output.
- About one-fifth of rows have no usable date, so no time-trend analysis is made; English only.
- Respect each site's terms of use and robots.txt, and do not redistribute downloaded content.

9. Run it and future work

Item	Detail
Stack	Python 3.10–3.12; requests, BeautifulSoup, trafilatura, pypdf, pandas, scikit-learn, matplotlib, reportlab, Playwright, pytest
Full run	python -m btli all (roughly 10–30 min for 200 rows)
Offline demo	python -m btli demo (synthetic data, runs in seconds)
Docker	docker build -t btli . && docker run --rm -v $PWD/output:/app/output btli

Next: embedding- or LLM-based topic classification compared against the human labels; paper-level TF-IDF or BERTopic for sub-themes; sentiment and hedging analysis; automatic robots.txt handling; a Streamlit dashboard.